

Guida SISTEMATICA Funzioni: Template da Usare SEMPRE

SCHEMA GENERALE PER OGNI FUNZIONE

1. ALGORITMO: pseudocodice preciso
 2. CORRETTEZZA: dimostrazione (induzione/invariante)
 3. COMPLESSITÀ: ricorrenza + risoluzione
 4. ESEMPIO: applicazione pratica
 5. VARIANTI: modifiche comuni
-

CATEGORIA 1: HEAP E ARRAY

1.1 VERIFICA PROPRIETÀ MAX-HEAP

ALGORITMO

pseudocode

```
IsMaxHeap(A, n):  
  for i = 1 to floor(n/2):  
    left = 2*i  
    right = 2*i + 1  
  
    if left ≤ n AND A[i] < A[left]:  
      return False  
    if right ≤ n AND A[i] < A[right]:  
      return False  
  return True
```

CORRETTEZZA

Per induzione su i: Controllo ogni nodo interno i che soddisfi $A[i] \geq A[2i]$ e $A[i] \geq A[2i+1]$. Se tutti i nodi interni rispettano la proprietà, l'intero array è un max-heap.

COMPLESSITÀ

$O(n)$ - visito ogni nodo interno una volta sola ($n/2$ nodi)

ESEMPIO

$A = [90, 65, 73, 45, 63] \rightarrow$ verifica $90 \geq 65, 73$ e $65 \geq 45, 63$ ✓

VARIANTI

- **Min-Heap:** cambia $<$ con $>$
 - **Con size separato:** IsMaxHeap(A, heap_size)
-

1.2 COSTRUZIONE MAX-HEAP DA ARRAY

ALGORITMO

pseudocode

```
BuildMaxHeap(A, n):
    heap_size = n
    for i = floor(n/2) down to 1:
        Max-Heapify(A, i)

Max-Heapify(A, i):
    l = 2*i
    r = 2*i + 1
    largest = i

    if l ≤ heap_size AND A[l] > A[largest]:
        largest = l
    if r ≤ heap_size AND A[r] > A[largest]:
        largest = r

    if largest ≠ i:
        swap A[i] with A[largest]
        Max-Heapify(A, largest)
```

CORRETTEZZA

Per induzione sul livello: Parto dalle foglie (già heap) e vado verso la radice. Ogni chiamata Max-Heapify(A,i) assume che i sottoalberi left(i) e right(i) siano già max-heap e rende max-heap il sottoalbero radicato in i.

COMPLESSITÀ

O(n) - non $O(n \log n)$! Somma geometrica: $\Sigma(\text{livello} \times \text{nodi_livello})$

ESEMPIO

[4,1,3,2,16,9,10,14,8,7] → [16,14,10,8,7,9,3,2,4,1]

1.3 INSERIMENTO IN HEAP

ALGORITMO

pseudocode

```
Heap-Insert(A, key):  
    heap_size = heap_size + 1  
    A[heap_size] = -∞  
    Heap-Increase-Key(A, heap_size, key)
```

```
Heap-Increase-Key(A, i, key):  
    A[i] = key  
    while i > 1 AND A[Parent(i)] < A[i]:  
        swap A[i] with A[Parent(i)]  
        i = Parent(i)
```

CORRETTEZZA

Invariante: $A[1..heap_size-1]$ è max-heap. Inserisco in posizione $heap_size$ e "faccio risalire" finché proprietà heap è ripristinata.

COMPLESSITÀ

$O(\log n)$ - nel caso pessimo risale fino alla radice (altezza heap)

CATEGORIA 2: BST E ALBERI

2.1 VERIFICA PROPRIETÀ BST (Metodo 1: In-Order)

ALGORITMO

pseudocode

```
IsBST(T):  
    is_valid, last = InOrderCheck(T.root, -∞)  
    return is_valid  
  
InOrderCheck(x, last):  
    if x = nil:  
        return True, last  
  
    valid_left, last = InOrderCheck(x.left, last)  
    if not valid_left OR last ≥ x.key:  
        return False, last  
  
    return InOrderCheck(x.right, x.key)
```

CORRETTEZZA

Proprietà BST: visita in-order produce sequenza ordinata. Se durante visita trovo elemento $\leq$ precedente $\rightarrow$ non è BST.

COMPLESSITÀ

O(n) - visita ogni nodo una volta

2.2 VERIFICA PROPRIETÀ BST (Metodo 2: Min-Max)

ALGORITMO

pseudocode

```
IsBST(T):  
    return CheckBST(T.root, -∞, +∞)  
  
CheckBST(x, min_val, max_val):  
    if x = nil:  
        return True  
  
    if x.key ≤ min_val OR x.key ≥ max_val:  
        return False  
  
    return CheckBST(x.left, min_val, x.key) AND  
        CheckBST(x.right, x.key, max_val)
```

CORRETTEZZA

Propagazione vincoli: ogni nodo deve rispettare $\min < \text{key} < \max$. Propago vincoli ai sottoalberi.

COMPLESSITÀ

O(n) - visita ogni nodo una volta

2.3 COSTRUZIONE BST BILANCIATO DA ARRAY

ALGORITMO

pseudocode

```
ArrayToBST(A, p, q):  
    if p > q:  
        return nil  
  
    m = floor((p + q) / 2)  
    x = CreateNode(A[m])  
    x.left = ArrayToBST(A, p, m-1)  
    x.right = ArrayToBST(A, m+1, q)  
    return x
```

CORRETTEZZA

Per induzione su n: Scelgo elemento mediano come radice. Per proprietà array ordinato: elementi a sinistra < radice < elementi a destra. Ricorsivamente costruisco sottoalberi bilanciati.

COMPLESSITÀ

$T(n) = 2T(n/2) + O(1) = \mathbf{O(n)}$ (Master Theorem)

ESEMPIO

$[1,3,5,7,9] \rightarrow \text{radice}=5, \text{left}=[1,3], \text{right}=[7,9]$

2.4 INSERIMENTO BST CON CAMPO AGGIUNTIVO

ALGORITMO (campo "even" = somma sottoalbero pari/dispari)

pseudocode

```
Insert(T, z):
    x = T.root
    y = nil
    z.even = (z.key mod 2 == 0)

    while x ≠ nil:
        x.even = (x.even == z.even) // XOR logico
        y = x
        if z.key < x.key:
            x = x.left
        else:
            x = x.right

    z.p = y
    if y = nil:
        T.root = z
    else if z.key < y.key:
        y.left = z
    else:
        y.right = z
```

CORRETTEZZA

Invariante: $x.\text{even}$ = (somma sottoalbero x è pari). Quando inserisco z, aggiornò tutti gli antenati perché la loro somma cambia.

COMPLESSITÀ

$O(h)$ dove h = altezza albero

2.5 RANGE QUERY IN BST

ALGORITMO

pseudocode

```
Range(T, k1, k2):
    RangeRec(T.root, k1, k2)

RangeRec(x, k1, k2):
    if x ≠ nil:
        if x.key ≥ k1:
            RangeRec(x.left, k1, k2)
        if k1 ≤ x.key ≤ k2:
            print x.key
        if x.key ≤ k2:
            RangeRec(x.right, k1, k2)
```

CORRETTEZZA

Potatura BST: vado a sinistra solo se $x.key \geq k1$, a destra solo se $x.key \leq k2$. Stampo $x.key$ solo se nell'intervallo.

COMPLESSITÀ

$O(h + k)$ dove h =altezza, k =elementi nell'intervallo

CATEGORIA 3: DIVIDE ET IMPERA

3.1 RICERCA BINARIA (Template Base)

ALGORITMO

pseudocode

```
BinarySearch(A, p, r, k):
    if p > r:
        return nil

    q = floor((p + r) / 2)
    if A[q] = k:
        return q
    elif A[q] > k:
        return BinarySearch(A, p, q-1, k)
    else:
        return BinarySearch(A, q+1, r, k)
```

CORRETTEZZA

Per induzione su lunghezza: Se array vuoto $\rightarrow$ nil. Altrimenti confronto con elemento centrale e ricorro sulla metà corretta per proprietà ordinamento.

COMPLESSITÀ

$T(n) = T(n/2) + O(1) = \mathbf{O(\log n)}$ (Master Theorem)

3.2 TROVA ELEMENTO $> x$ (Variante Ricerca)

ALGORITMO

pseudocode

```
FindGreater(A, p, r, x):  
    if p > r:  
        return r + 1  
  
    q = floor((p + r) / 2)  
    if A[q] ≤ x:  
        return FindGreater(A, q+1, r, x)  
    else:  
        return FindGreater(A, p, q-1, x)
```

CORRETTEZZA

Invariante: soluzione è in $A[p..r]$ o non esiste. Se $A[q] \leq x$ cerco a destra, altrimenti la soluzione è $A[q]$ o a sinistra.

COMPLESSITÀ

$O(\log n)$

3.3 TROVA GAP IN ARRAY

ALGORITMO

pseudocode

```
// Precondizione: A[r] - A[p] ≥ r - p + 1  
FindGap(A, p, r):  
    if p = r - 1:  
        return p  
  
    q = floor((p + r) / 2)  
    if A[q] - A[p] > q - p:  
        return FindGap(A, p, q)  
    else:  
        return FindGap(A, q, r)
```

CORRETTEZZA

Invariante: esiste gap in $A[p..r]$. Se differenza $A[q]-A[p] >$ spazio disponibile $\rightarrow$ gap a sinistra, altrimenti a destra.

COMPLESSITÀ

$$T(n) = T(n/2) + O(1) = \mathbf{O(\log n)}$$

3.4 TROVA MASSIMO (Divide et Impera)

ALGORITMO

pseudocode

```
FindMax(A, p, r):  
    if p = r:  
        return A[p]  
  
    q = floor((p + r) / 2)  
    max_left = FindMax(A, p, q)  
    max_right = FindMax(A, q+1, r)  
    return max(max_left, max_right)
```

CORRETTEZZA

Per induzione: Caso base un elemento. Altrimenti massimo è il max tra massimi delle due metà.

COMPLESSITÀ

$$T(n) = 2T(n/2) + O(1) = \mathbf{O(n)} \text{ (Master Theorem)}$$

3.5 CONTA INVERSIONI

ALGORITMO

pseudocode

```
CountInversions(A, p, r):  
    if p ≥ r:  
        return 0  
  
    q = floor((p + r) / 2)  
    inv_left = CountInversions(A, p, q)  
    inv_right = CountInversions(A, q+1, r)  
    inv_cross = MergeAndCount(A, p, q, r)  
    return inv_left + inv_right + inv_cross  
  
MergeAndCount(A, p, q, r):  
    // Come merge di MergeSort ma conta inversioni cross  
    inversions = 0  
    // quando prendo da right, aggiungo (left_remaining) a inversions
```

CORRETTEZZA

Divide et impera: inversioni totali = inv(sinistra) + inv(destra) + inv(cross). Durante merge conto inversioni cross.

COMPLESSITÀ

$T(n) = 2T(n/2) + O(n) = O(n \log n)$

TEMPLATE UNIVERSALE CORRETTEZZA

Per Algoritmi Ricorsivi:

1. **Caso base:** dimensione piccola ($n=0,1$)
2. **Passo induttivo:** se vale per input $< n$, vale per n
3. **Riduzione:** ogni chiamata riduce problema

Per Algoritmi Iterativi:

1. **Inizializzazione:** invariante vera all'inizio
2. **Mantenimento:** invariante preservata da ogni iterazione
3. **Terminazione:** invariante + condizione stop → correttezza

Schema Template Completo:

FUNZIONE: nome(parametri)

INPUT: descrizione precondizioni

OUTPUT: descrizione postcondizioni

ALGORITMO:

[pseudocodice]

CORRETTEZZA:

[dimostrazione per induzione/invariante]

COMPLESSITÀ:

[ricorrenza → risoluzione → risultato finale]

ESEMPIO:

[applicazione concreta]

USA SEMPRE QUESTO SCHEMA PER OGNI ESERCIZIO!